FINAL REPORT

VendorPulse AI

Team: TEAM012 · Member: V V S S N Padmaji Saladi (2025mb26046) · Instructor: Vinaya Sathyanarayana

1. Product Strategy and Opportunity Framing

Product vision

Every store knows which vendor is coming, when, and how far to trust that, so the dock is staffed when the truck arrives and nobody waits for a truck that is not coming.

Jobs to be done

- **Receiving lead:** When I plan the dock schedule each morning, I want to know which small vendors will really arrive and when, so that I staff the dock at the right times and stop vendors from self-scanning.
- **Receiving associate:** When a vendor has not shown up or a truck appears unexpectedly, I want a quick answer on my handheld, so that I do not have to call the office or search several systems.
- **Vendor manager:** When I review local vendors, I want a fair, evidence-backed reliability view, so that I can start a factual conversation with a vendor who keeps missing windows.

Opportunity framing

- **Why it matters:** DSD receiving is a daily, repeated decision in every store. Errors show up as idle labor, self-scan variances, wrong balance-on-hand and lost sales.
- **Why current alternatives are weak:** agreed windows are not followed by NEX vendors; spreadsheets and reports describe the past and cannot read emails or call notes; knowledge leaves with experienced leads.
- **Why AI helps:** arrival and reschedule behavior is learnable from history (vendor, weekday route, season, holidays). Language models make the answer usable at the dock and turn unstructured messages into proposed updates.

AI opportunity matrix

AI idea	Business value	Feasibility now	Priority
Arrival window + late / reschedule risk for NEX vendors	High: staffing, shrink	High: history exists	MVP
Vendor reliability score with evidence	High: vendor management	High: rule on history	MVP
Natural-language Q&A on handheld	Medium-High: adoption	High: API LLM	MVP
Read vendor emails / dock notes into proposed changes	Medium	Medium	MVP (human confirms)
External signals (weather, traffic) + proactive alerts	Medium	Medium	Phase 2
PO, items and delivery variance answers	High	Low: needs PO/receiving integration	Phase 3-4
Auto-drafted vendor performance letters	Medium	Medium; governance heavy	Phase 5

Feature prioritization (RICE)

Reach = store users per week in the pilot store; Impact 0.5-3; Confidence %; Effort in person-weeks. Values are team estimates.

Feature	Reach	Impact	Confidence	Effort	RICE
Today's schedule with predicted window + risk	12	3	80%	3	9.6
Evidence and confidence on every answer	12	2	90%	1	21.6
"Not enough data" fallback	12	2	90%	0.5	43.2
Reliability score (0-100)	6	2	70%	1.5	5.6
Natural-language Q&A	12	1.5	70%	3	4.2
Email / dock-note extraction with confirm	4	1.5	60%	3	1.2
Weather / traffic signals	12	1	50%	4	1.5 (Phase 2)

The trust features score highest because they are cheap and the pilot fails without them. Q&A ranks below the schedule view: the core value is the prediction; chat is how it reaches the dock.

Build vs buy vs partner/API

- **Recommended path:** API-first LLM (Azure OpenAI) with tool calling, plus custom ML models built on Azure ML from the retailer's delivery history.
- **Why:** no off-the-shelf DSD tool predicts NEX vendor behavior for this retailer; the differentiator is the retailer's own data and workflow fit. Building a language model adds cost and no advantage. Azure fits the retailer's existing Microsoft stack and data governance.
- **Trade-offs:** dependence on a model vendor's pricing and versions (mitigated by keeping the LLM thin and swappable behind tool APIs); in-house ML needs MLOps ownership (retraining, drift monitoring).
- **Platform vs feature:** VendorPulse starts as a feature inside the existing store associate app, not a separate app. It becomes a DSD platform only when Phases 3-5 add PO, item and variance tools to the same copilot.

First wedge

One pilot store, 20-30 small and local NEX vendors, three questions only: when will each vendor arrive, how likely are they to be late or reschedule, and how reliable have they been. No purchase orders, items, variances or vendor penalties.

Course case lens

Case	Similar	Different	What we transfer / do differently
Swiggy (invisible AI, ETA)	Predicting an arrival time that drives someone else's planning	Swiggy controls its riders and has live GPS; we do not control vendors and have no live location in the MVP	Show a window, not a point time; keep AI mostly invisible inside the existing app; add live signals only in Phase 2
Instacart / Tesla (lifecycle, drift)	Behavior shifts with seasons, routes and shocks	Smaller data per vendor; drift is per vendor	Weekly retraining, per-vendor drift alerts, fall back to agreed window when data is stale

Case	Similar	Different	What we transfer / do differently
Apple Card / bias in lending	A score can disadvantage a group unfairly	Vendors, not consumers; harm is commercial	Minimum history before any score, visible components, vendor dispute path, no automated action from the score

2. PM Artifact Pack

The full PRD-lite, roadmap and metric definitions are in the Appendix. This section summarizes the decisions.

Problem statement

Receiving leads at a 300-store supercenter chain cannot reliably plan dock labor for small and local NEX vendors, because these vendors arrive outside their agreed windows and reschedule without notice. The decision we improve is the morning staffing plan and the response when a vendor changes plans.

PRD summary

Item	Summary
User problem	No reliable view of which NEX vendor is coming today, when, and how dependable they are.
Core workflow	Lead opens today's list → sees predicted window + risk → asks why for risky vendors → plans staffing → confirms or rejects AI-read vendor messages → actual arrival is logged → models retrain weekly.
AI role	ML predicts arrival time, late risk and reschedule risk; a transparent rule computes the reliability score; the LLM understands questions, calls these tools and explains results; the LLM extracts proposed changes from messages.
Data assumptions	At least 12 months of receiving timestamps per store; vendor master and agreed schedules are accurate; reschedules and no-shows can be inferred from schedule vs receipt records.
Risks	Wrong prediction leads to wrong staffing; invented answers; unfair scores for small vendors; sensitive vendor data exposure; drift.
Fallback logic	Fewer than 8 past deliveries → "not enough data", plan to agreed window. Low confidence → show agreed window plus a warning. LLM or model unavailable → plain schedule list. Unclear message → route to lead, no change.

MVP definition

The MVP must prove that (1) predictions beat the agreed window and a simple vendor average, (2) store users understand and trust the risk bands and evidence, and (3) humans can correct the AI quickly. MVP features: today and 7-day schedule with predicted window and risk, reliability score, last-90-day history, explanations, confidence, "not enough data" response, message extraction with confirm/reject, and a manual override with reason.

Roadmap with proof gates

Phase	Scope	Proof gate to move on
0. Prototype (done)	Simulated data, ML models, copilot UI, HITL	Workflow and trust model are clear; model beats baselines on test data
1. MVP pilot (10 weeks, 1 store)	Real NEX history, live copilot on handhelds	≥80% arrivals within ±45 min; ≥90% grounded answers, 0 invented numbers; unstaffed arrivals and self-scans down ≥25%
2. Better predictions (5-10 stores)	Weather, traffic, holidays, proactive alerts	Accuracy holds across store types; alert precision ≥60%
3. Purchase orders	PO and expected items per delivery	PO data quality; answers correct ≥95%
4. Delivered items and variances	Receiving data, digitized NEX invoices	Variance detection validated by Inventory Control
5. Vendor performance	Accuracy, shrink, chargebacks; drafted vendor letters	Governance sign-off; human approval on every vendor communication

Success metrics

- **Product:** share of mornings the lead uses VendorPulse to plan; questions answered without escalation.
- **Model:** arrivals within ±45 min of prediction; ROC-AUC and precision/recall for late and reschedule risk.
- **Business:** unstaffed vendor arrivals per week; vendor self-scan events; receiving idle hours; count variances.
- **Trust / guardrail:** grounded-answer rate; invented numbers (must be zero); override rate; extraction errors caught at confirmation.

How these artifacts change because the product is AI-powered

Artifact	What changed for VendorPulse
Problem statement	Names the decision being improved (morning staffing plan), not just a pain point.
User stories	Include what the user sees when the model is unsure or has no data, and the cost of a wrong prediction.
PRD	Adds data assumptions, minimum-history rule, confidence logic and fallbacks.
Roadmap	Phases are proof gates with thresholds, not only feature releases.
Metrics	Model metrics (accuracy, AUC) are tied to business metrics (unstaffed arrivals, self-scans).
Launch checklist	Requires review, rollback to plain schedule, and an escalation path.
Pricing note	Counts inference and MLOps cost per store per day.

3. UX, Workflow and Trust Design

Main workflow

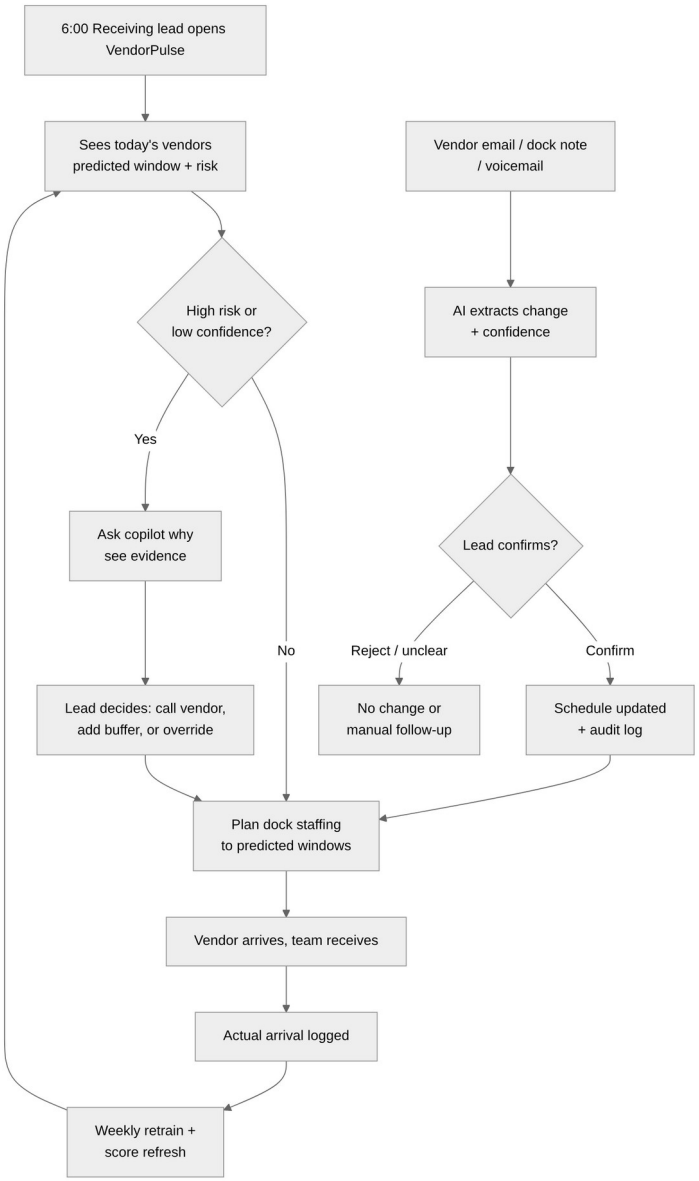


Figure 1. Receiving lead's daily loop with VendorPulse. Decisions stay with the lead.

Key screens (from the working prototype)

- **Today:** vendors sorted by predicted arrival, each with window, risk chip, confidence, three evidence lines and an Override button.
- **Ask the copilot:** answers show the tools called, the data used and a confidence level.
- **Vendor messages:** original text next to the extracted change; Confirm / Reject; low-confidence items can only be sent to the lead.
- **Vendor scores:** 0-100 score with its four components; "not enough data" for new vendors.
- **Model evidence:** how the model compares with the agreed window and a simple average.

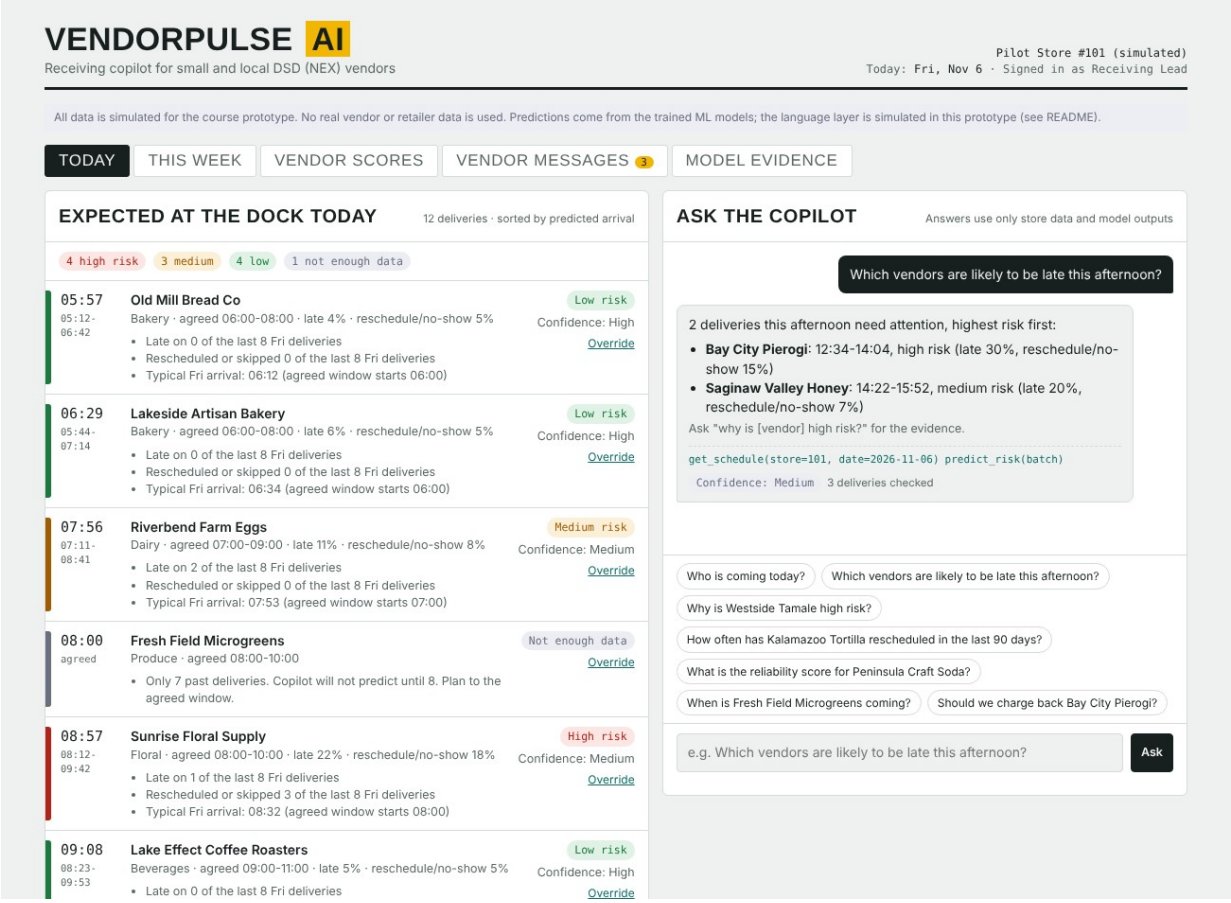


Figure 2. Today view and copilot (prototype, simulated data). More screens in Appendix/Screenshots_or_Video.

Trust and explainability

What the user needs to know	Must be visible	Should be hidden
When the vendor will probably arrive and how sure we are	Predicted window, risk band, probability, confidence, "late on 5 of last 8 Fridays"-style evidence	Model internals, feature weights, raw probabilities beyond 2 digits
Whether a score is fair	Score components and the number of deliveries behind it	Scores from dock associates' view (leads and Merchandising only)
What the AI read from a message	Original message beside the extracted fields	Sender personal details beyond what is needed

Uncertainty handling

- **High confidence** (8+ same-weekday deliveries, stable arrival times): show the predicted window and risk directly.
- **Medium / low confidence:** show the window with a confidence tag and evidence; recommend a buffer for High-risk vendors.
- **Not enough data** (fewer than 8 deliveries): no prediction or score; plan to the agreed window. The copilot says so instead of guessing.
- **Failure / fallback:** if the LLM or model service is down, the app shows the plain agreed schedule. Out-of-scope requests (POs, chargebacks) get a clear refusal that points to the owner.

Human-in-the-loop

- The lead decides staffing; predictions only inform it.
- Any schedule change read from an email, note or voicemail needs a person to confirm it. Nothing is applied automatically.
- Users can override any prediction with a reason ("vendor called, arriving 10:30"); overrides are logged and used to review the model.
- Scores are reviewed by the lead before they reach Merchandising; business actions on vendors stay with Merchandising and Vendor Management.

4. Data, Model and Evaluation Strategy

Problem as AI tasks

Task	Type	Method in prototype
Arrival time for a scheduled delivery	Regression (forecast)	Gradient-boosted regressor (absolute-error loss); window = prediction ±45 min
Late arrival (after the agreed 2-hour window)	Binary classification	Logistic regression
Reschedule or no-show	Binary classification	Logistic regression
Vendor reliability score	Scoring rule	40% on-time, 25% reschedule, 20% no-show, 15% count variance, last 90 days
Answer questions	Generation with tool calling (RAG over structured data)	Simulated by an intent router; production = Azure OpenAI
Read vendor messages	Information extraction	Simulated (pre-filled extractions); production = LLM with confidence

Data sources

- **Internal:** receiving transaction timestamps and status, vendor master, agreed delivery days and windows, reschedule records, count variances at receiving.
- **External (Phase 2):** weather, traffic, holidays, local events.
- **User-generated:** vendor emails, dock notes, voicemail transcripts, overrides with reasons.

Data assumptions and the simulated dataset

Real retailer data cannot be used in a course submission, so the prototype uses a simulated history for one store: 25 fictional NEX vendors, 18 months (May 2025 to Nov 2026), 3,098 past scheduled deliveries plus 42 scheduled for the coming week. The simulation encodes patterns observed in NEX receiving: each vendor has its own habitual arrival offset per weekday route, one weekday that runs late, more delay and rescheduling around holidays and in winter, and different reliability profiles. Two vendors were onboarded recently to test the "not enough data" path. The real pilot must confirm these patterns.

Labels / ground truth

Labels come from the receiving system, so no manual labeling is needed: actual arrival time minus agreed window start gives the arrival offset; arrival after the window end = late; a scheduled delivery with no receipt and a later receipt within 3 days = rescheduled; no receipt = no-show. In the pilot, receiving leads confirm ambiguous cases weekly, and confirmed message extractions become labeled examples for the extraction prompt.

Baselines

- **Agreed-window rule:** assume the vendor arrives at the start of the agreed window (today's practice).
- **Vendor recent average:** median of the vendor's last 10 arrivals, and the vendor's 90-day late / reschedule rate as risk (what a dashboard would show).

Model / API choice

Logistic regression was chosen for the risk models because it matched gradient boosting on the test set (late AUC 0.761 vs 0.741; reschedule AUC 0.705 vs 0.586) and is easier to explain and audit. Gradient boosting is used for arrival time, where it was more accurate. The reliability score is a rule on purpose: vendors and managers must be able to see exactly how it is calculated. The LLM is API-based and never computes numbers.

Evaluation results (simulated data, time-based split)

Training: 2025-05-19 to 2026-05-30 (2019 rows). Test: 2026-06-01 to 2026-11-05 (884 rows).

Metric	Agreed-window rule	Vendor recent average	ML model
Arrivals within ±45 min	49%	63%	70%
Mean absolute arrival error	70.7 min	47 min	41.5 min
Late risk ROC-AUC	n/a	0.719	0.761
Reschedule / no-show ROC-AUC	n/a	0.635	0.705
Top 20% flagged: precision / recall	n/a	54% / 40%	57% / 41%

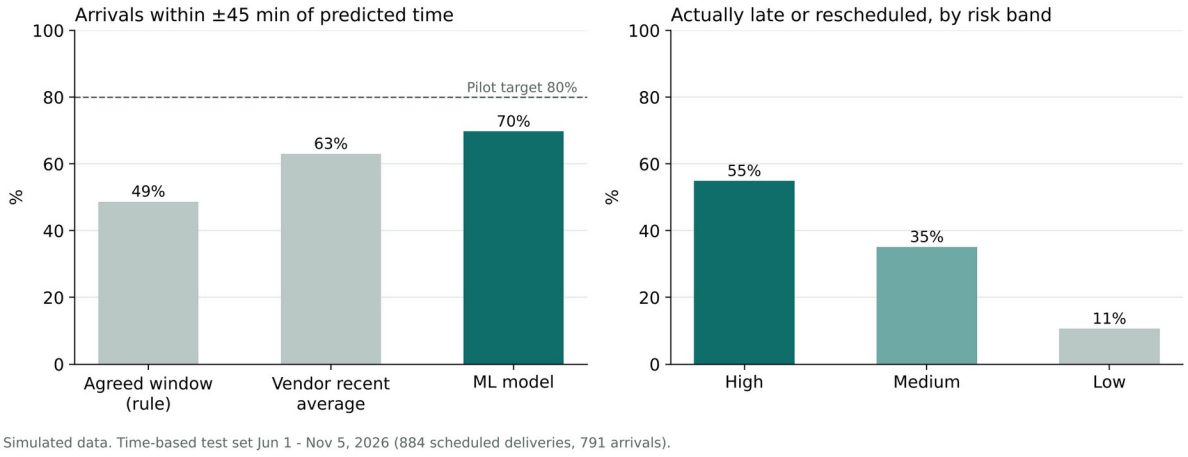


Figure 3. Model vs baselines. Right: High-risk deliveries were 5x more likely to have a problem than Low-risk ones.

What this shows: the model clearly beats the agreed-window rule and moderately beats a simple vendor average. The risk bands separate deliveries well. **What it does not show:** real-world accuracy. Arrival accuracy (70%) is below the 80% target, so the pilot is needed and Phase 2 signals may be required.

Evaluation framework

Metric type	Metric	Why it matters
Technical	% arrivals within ±45 min; MAE; ROC-AUC; precision/recall of High band	Staffing plans depend on the window being right

Metric type	Metric	Why it matters
Technical (LLM)	Grounded-answer rate on a 100-question test set; extraction precision	Wrong numbers on a handheld lead to wrong actions
Business	Unstaffed arrivals/week; self-scan events; idle receiving hours; count variances	These are the cost and shrink drivers
Trust / guardrail	Invented numbers (must be 0); override rate; % answers with evidence; user trust survey	Adoption fails without trust; high overrides signal model problems

Launch thresholds (pilot → scale)

- Arrivals within ±45 min ≥80% on real pilot data, and better than the vendor-average baseline.
- Late-risk ROC-AUC ≥0.70; High-band problem rate at least 3x the Low band.
- ≥90% of test questions answered correctly and grounded; zero answers with invented numbers.
- Extraction precision ≥95% on confirmed changes; nothing applied without confirmation.
- Unstaffed arrivals and self-scan events down ≥25% versus the 4-week pre-pilot baseline; ≥70% of mornings planned with the tool.

5. Business, Economics and Scaling

Monetization model

Internal value case: VendorPulse is an internal tool, funded from store operations savings, not sold. (If offered by an implementation partner, it could be priced per store per month.)

Value logic (hypotheses to measure in the pilot)

All inputs below are assumptions for a single store. They are not measured results.

Driver	Assumption	Annual value per store
Less idle waiting for late / no-show vendors	42 NEX deliveries/week; 28% late or rescheduled; 30 min wasted each; tool avoids half; $20/hr loaded labor	≈ $3,000
Fewer self-scan variances	2 self-scan events/week at ~$40 shrink each; tool avoids half	≈ $2,100
Correct balance-on-hand, faster replenishment	Not quantified in the MVP	Upside
Total		≈ $5,100 / store; ≈ $1.5M across 300 stores

Cost drivers

Cost	Assumption at 300 stores	Annual cost
LLM API (questions)	40 questions/store/day; ~$0.003 per question (to verify against current Azure OpenAI pricing)	≈ $13,000
LLM API (message extraction)	10 messages/store/day; ~$0.003 each	≈ $3,300
ML training and hosting	Weekly retrain, batch scoring each night	≈ $20,000-40,000
Data / MLOps and support	~1 FTE data/ML engineer, part-time product owner	≈ $150,000-200,000

Cost	Assumption at 300 stores	Annual cost
Human review	Lead confirms 1-3 messages/day, about 5 min	Absorbed in existing role

Unit economics view

Running cost is roughly $2 per store per day against an assumed value of about $14 per store per day. Inference is a small share of cost; people (data engineering, MLOps, support) dominate. The case breaks even if the pilot shows about 15% of the assumed value. Keep the LLM thin: models score in batch overnight, so each question is a cheap lookup plus a short generation.

Scaling risks

- **Latency:** answers must return in under 3 seconds on store Wi-Fi; precompute predictions nightly and cache today's list on the device.
- **Cost growth:** cap tokens per answer, use a small model for routine questions, monitor cost per store per day.
- **Workflow complexity:** stores differ in dock layout and staffing; roll out in waves by store type, as with current app releases.
- **Governance burden:** vendor scores across 300 stores need a review owner and a dispute process before Merchandising uses them.

6. Ethics, Governance and Risk Note

Main risks

- **Unfair vendor scores (bias):** small local vendors may look unreliable because of their size or lack of digital tools, not performance.
- **Hallucinated or wrong answers:** a wrong time or score leads to poor staffing or an unfair vendor conversation.
- **Privacy and commercial sensitivity:** vendor performance and schedules are confidential; messages contain personal contact details.
- **Misuse:** using scores to penalize vendors automatically, or to evaluate associates.
- **Over-reliance and drift:** staff stop applying local knowledge; vendor behavior changes over seasons and routes.

Harm scenarios

- A local vendor gets a low score from few deliveries and loses shelf space without a fair hearing.
- The copilot states a wrong arrival time; the dock is unstaffed and the vendor self-scans, creating shrink.
- A dock associate sees another vendor's performance data and shares it.

Controls

- **Guardrails:** the LLM answers only from tool results; numbers must come from tools; refuses out-of-scope topics (POs, chargebacks, penalties); minimum 8 deliveries before any prediction or score.
- **Review process:** human confirmation for every schedule change from messages; lead reviews scores before sharing; vendors can see and dispute their score components.
- **Escalation path:** low-confidence extractions go to the receiving lead; repeated low scores go to Vendor Management, who decide any action.

- **Access:** role-based access; dock associates see schedule and risk only; scores visible to leads and Merchandising.
- **Logging / auditability:** every answer logs the question, tools called, data version and model version; every override and confirmation is logged with user and reason.
- **Monitoring:** weekly accuracy and drift report per vendor; score distribution checked by vendor size.

Launch blockers

- Any answer in the test set that contains a number not returned by a tool.
- Arrival accuracy below the vendor-average baseline on real pilot data.
- Scores for small vendors systematically lower than their on-time data supports.
- No named owner for score review and vendor disputes.

7. Prototype / MVP

Item	Detail
Prototype type	Web app (single page, works on a handheld screen) + Python ML pipeline. LLM layer is Wizard-of-Oz: an intent router stands in for Azure OpenAI so the demo runs without an API key.
What it demonstrates	The morning planning loop: today's vendors with predicted window and risk, evidence, confidence, "not enough data", vendor scores, Q&A with tool trace, message extraction with confirm/reject, and override with reason.
AI role	Real trained ML models produce every prediction; the LLM role (understanding and explaining) is shown through the tool-calling pattern.
What it does not yet prove	Accuracy on real data; real LLM answer quality and hallucination rate; handheld performance on store Wi-Fi; integration with the receiving system; user adoption.
Links	Live demo: https://claude.ai/artifact/7Si4qbkxUE6fwvSwYqg6vv (share before submitting). Code: Prototype folder (README explains how to run). Screenshots: Appendix/Screenshots_or_Video.

8. Recommendation

Pilot. The problem is real, repeated and measurable, and AI is the right lever because the decision depends on predicting vendor behavior, not listing it. The prototype shows the model beats today's rule, the risk bands are meaningful and the human-in-the-loop design is practical. Accuracy on simulated data is below the target, so we recommend a 10-week, one-store pilot on real data with the launch thresholds in Section 4. Scale to more stores only if those thresholds are met; stop if the model cannot beat the vendor-average baseline on real data.